

# FRIDAY — Autonomous Build Playbook

Copy-paste prompts, skill invocations, and per-technology implementation notes that let Claude Code build the entire FRIDAY application from this document — phase by phase, test-driven, with verification gates.

Prepared for: Ethan Soliven · Date: July 11, 2026 · Companion to: FRIDAY Implementation Plan v0.2

---

## 1. How to Use This Document

- **Setup (you, once, ~5 min):** create the repository yourself (GitHub or local) and upload the two PDFs at their proper paths — that's the entire manual scaffold. Claude Code builds everything else around them:

### One-time setup — you create the repo and place the files

```
1. Create the repository (e.g. github.com -> New repository 'friday', private).
2. Upload the two documents at exactly these paths:
docs/FRIDAY-Second-Brain-Implementation-Plan.pdf (the spec, v0.2)
docs/FRIDAY-Autonomous-Build-Playbook.pdf (this document)
3. Clone it and start Claude Code inside it:
git clone https://github.com/<you>/friday && cd friday
claude # logged in with your subscription
```

The repo already being git-initialized is expected — Claude Code must NOT re-init it; it commits into the existing history and pushes at each phase end.

- **Kickoff (you, once):** paste the Master Prompt (§2) into Claude Code. It reads both PDFs, creates docs/BUILD\_STATE.md, and starts Phase P0.
- **Per phase:** Claude Code works autonomously through the phase using the prompts in §5 (it reads them from this PDF itself). You only show up at the end of each phase to eyeball the demo and say *continue* — or paste the next phase prompt yourself if you prefer manual gating.
- **Resume anytime:** new session? Paste the Resume Prompt (§7). BUILD\_STATE.md carries the position, so no context is lost between sessions.

**Why this works:** the spec PDF says *what* to build; this playbook says *how* — exact libraries, skill workflow, invariants, and machine-checkable exit criteria. Claude Code fills in the rest, which is what it is good at.

## 2. The Master Kickoff Prompt

Paste this verbatim as your first message. It establishes the mission, the skill workflow, and the rules that hold for the entire build.

## MASTER PROMPT — paste into Claude Code

You are building Project FRIDAY, my local second-brain assistant, end to end.

REPO STATE: this repository already exists (I created it) and contains docs/ with the two PDFs below. Verify both files exist before anything else; if either is missing or at the wrong path, STOP and tell me — do not proceed from memory. Do NOT run git init; commit into the existing history (conventional commits) and `git push` at the end of every phase after its exit criteria pass.

READ FIRST (in full, before any action):

1. docs/FRIDAY-Second-Brain-Implementation-Plan.pdf — the spec (v0.2): architecture, vault template, RAG design, agent tools, suggest-then-approve loop, dashboard, roadmap.
2. docs/FRIDAY-Autonomous-Build-Playbook.pdf — this playbook: Section 5 has one prompt per phase; Section 6 has the technology cheat sheet; Section 4 lists the skills to use.

MISSION: implement the roadmap phases P0 -> P1 -> P1.5 -> P2 -> P3 -> P4 exactly as specified. Work autonomously. Do not ask me questions a spec reader could answer; decide and log the decision in docs/BUILD\_LOG.md. Only stop for: OAuth credentials I must create, or a spec contradiction you cannot resolve.

WORKFLOW FOR EVERY PHASE (non-negotiable):

1. Invoke Skill superpowers:writing-plans -> write the phase plan to docs/plans/.
2. Invoke Skill superpowers:subagent-driven-development to execute the plan; every task follows superpowers:test-driven-development (test first, watch it fail, implement, watch it pass). Commit after each green task.
3. Before declaring the phase done: invoke superpowers:verification-before-completion, run the phase's EXIT CRITERIA block from Playbook Section 5, paste real command output.
4. Run /code-review on the phase's diff; fix CONFIRMED findings; commit.
5. Update docs/BUILD\_STATE.md (phase, task, status, next action) and docs/BUILD\_LOG.md.

HARD INVARIANTS (violating any of these is a bug, no matter what else works):

- I1. The ONLY module that writes to the vault, Google Calendar, or Todoist is backend/writer.py. Everything else proposes rows into the suggestions table. Exception: study outputs may create NEW files under 15-Courses/\*/study/ only.
- I2. writer.py git-commits the vault before every apply (auto-undo).
- I3. 99-Private/ and #no-ai notes never enter the index or any prompt.
- I4. No secrets in the repo: tokens go to the OS keyring, config to .env (gitignored).
- I5. Agent auth = my Claude Code subscription login. Do NOT set ANTHROPIC\_API\_KEY.
- I6. Every RAG-grounded answer and every generated exam question carries a source citation (note path or 'file p.N / slide N'); drop uncited exam questions.

STACK (pinned; details in Playbook Section 6): Python 3.12 + FastAPI + uvicorn; claude-agent-sdk (model claude-opus-4-8); chromadb + sentence-transformers bge-small-en-v1.5; SQLite; watchdog; pdfplumber / python-pptx / python-docx; Next.js 14 + TypeScript + Tailwind; pytest + Playwright.

Begin now: create docs/BUILD\_STATE.md and docs/BUILD\_LOG.md, then execute Phase P0 per Playbook Section 5.

## 3. Repository Layout & Conventions

Target repository structure (docs/ exists already — Claude Code creates the rest in P0)

```
friday/ # your existing repo, cloned
backend/
main.py # FastAPI app: REST + /ws/chat WebSocket
agent/ # Claude Agent SDK runtime, tool definitions, prompts/
rag/ # watcher.py, extract.py, chunk.py, embed.py, retrieve.py
connectors/ # gcal.py, todoist.py, obsidian.py (URI + vault paths)
tasks/ # markdown task parser (Obsidian Tasks syntax)
study/ # study_guide.py, practice_exam.py, grader.py, syllabus.py
suggestions.py # queue model + API
writer.py # THE single write path (I1) + git auto-commit (I2)
db.py # SQLite (sqlite3, WAL mode)
web/ # Next.js 14 app (pages: today, tasks, chat, study, review, insights)
vault-template/ # the PARA + 15-Courses skeleton copied on first run
tests/ # pytest; tests/e2e/ Playwright
docs/ # both PDFs (uploaded by me), BUILD_STATE.md, BUILD_LOG.md, plans/
.env.example .gitignore README.md pyproject.toml
```

- **Conventions:** type hints everywhere; Pydantic models for every API payload; conventional commits (feat(rag): ...); each backend package has a matching test file.
- **Config:** .env holds VAULT\_PATH, FRIDAY\_DB, ports; secrets (Google/Todoist tokens) go to the OS keyring via the keyring package (I4).

4. Skill Workflow Map

Which Claude Code skill to invoke when — the Master Prompt enforces this; the table is the reference.

| Stage                        | Skill / command                            | Purpose                                                                      |
|------------------------------|--------------------------------------------|------------------------------------------------------------------------------|
| Start of each phase          | superpowers:writing-plans                  | Turn the phase prompt + spec section into a step-by-step plan in docs/plans/ |
| Executing the plan           | superpowers:subagent-driven-development    | Dispatch plan tasks to subagents in-session with review between tasks        |
| Every code task              | superpowers:test-driven-development        | Red-green-refactor; no implementation before a failing test                  |
| Any bug / failing test       | superpowers:systematic-debugging           | Root-cause before fixing; prevents thrash loops                              |
| Dashboard UI work (P2-P4)    | frontend-design + ui-ux-pro-max            | Distinctive dashboard styling instead of default-template look               |
| Before claiming a phase done | superpowers:verification-before-completion | Run exit criteria; evidence before assertions                                |
| End of each phase            | /code-review (medium)                      | Correctness review of the phase diff; fix CONFIRMED findings                 |
| Risky/parallel experiments   | superpowers:using-git-worktrees            | Isolate big refactors from the main working tree                             |

5. Phase Prompts

The Master Prompt tells Claude Code to read these from this PDF. Each is self-contained: objective, implementation directives, and a machine-checkable exit-criteria block.

## PHASE P0 — Foundation

OBJECTIVE: runnable skeleton + vault template generator.  
PRE-FLIGHT: confirm docs/ holds both PDFs (spec + playbook); create .gitignore FIRST (node\_modules, .env, .friday/, \_\_pycache\_\_, demo-vault/) so nothing junk gets pushed.  
BUILD:  
- pyproject.toml (Python 3.12) with fastapi, uvicorn[standard], pydantic, watchdog, chromadb, sentence-transformers, pdfplumber, python-pptx, python-docx, keyring, python-frontmatter, claude-agent-sdk, pytest, httpx. Web: create-next-app@14 (TS, Tailwind).  
- vault-template/ exactly per spec Section 3 (00-Inbox .. 99-Private, 15-Courses/ sample course CS000 with course.md, notes/, materials/, study/). `friday init <path>` CLI copies it, git-inits the vault, writes VAULT\_PATH to .env.  
- backend/main.py: GET /health, GET /api/notes (walk vault, return paths+titles).  
- db.py: SQLite at .friday/friday.db, WAL mode; suggestions table: id, created\_at, kind(schedule|task|note), payload\_json, rationale, status, applied\_at.  
- web/: layout shell with nav (Today/Tasks/Chat/Study/Review/Insights), pages stubbed; /api proxy to :8000.  
EXIT CRITERIA (run and paste output):  
pytest -q -> all pass (init CLI + /api/notes tests)  
friday init ./demo-vault && ls demo-vault -> template folders present, .git exists  
curl localhost:8000/health -> {"status":"ok"}  
web: npm run build -> compiles; nav renders 6 pages

## PHASE P1 — RAG + chat (read-only agent)

OBJECTIVE: index the vault (markdown + course materials) and chat over it with citations.  
BUILD:  
- rag/watcher.py: watchdog observer on VAULT\_PATH, 2s debounce, queue -> reindex changed file; `friday reindex` full rebuild. Skip 99-Private/ and frontmatter no-ai (I3).  
- rag/extract.py: .md via frontmatter+text; .pdf pdfplumber (page numbers); .pptx python-pptx (slide numbers); .docx python-docx. Emit (text, meta) blocks.  
- rag/chunk.py: split on headings, ~350 tok target, 50 overlap; meta: path, title, heading, date, tags, wikilinks, course (from 15-Courses/<CODE>/ path), page/slide.  
- rag/embed.py: sentence-transformers 'BAAI/bge-small-en-v1.5', normalize\_embeddings=True, batch 64; chromadb.PersistentClient('.friday/chroma'), collection 'vault', ids = sha1(path+chunk\_idx) for idempotent upserts; delete-by-path before re-add.  
- rag/retrieve.py: vector top-20 + rank\_bm25 top-20 -> reciprocal-rank fusion -> recency boost score \*= exp(-age\_days/45) for 10-Daily and 00-Inbox docs -> top-8. Optional filters: course, tags, date range. Expand 1 hop of wikilinks (title lines).  
- agent/: claude-agent-sdk ClaudeSDKClient, model claude-opus-4-8, NO api key (I5 -- it inherits my Claude Code login). In-process MCP tools via @tool + create\_sdk\_mcp\_server: search\_vault, read\_note, list\_tasks(stub). System prompt in agent/prompts/chat.md: answer ONLY from tool results; cite [path] or [file p.N] for every claim; say 'not in your notes' when retrieval is empty.  
- main.py /ws/chat: bridge SDK streaming deltas -> WebSocket; web/chat page: streaming UI, citation chips -> obsidian://open?vault=...&file=... links.  
EXIT CRITERIA:  
pytest -q -> pass incl.: private-note never indexed; pdf chunk carries page meta; retrieval returns seeded fact; ws streams >1 delta chunk.  
Manual: seed demo vault w/ 3 notes + 1 PDF; ask a question answerable only from the PDF -> answer cites [<file> p.N]; ask something absent -> 'not in your notes'.

## PHASE P1.5 — Coursework engine

OBJECTIVE: study guides, practice exams with quiz mode, syllabus import.

BUILD:

- study/study\_guide.py: generate\_study\_guide(course, scope) -> RAG scoped to course -> agent drafts: outline, key concepts (cited), worked examples, gap list (topics in materials/ with no matching notes/ chunks). Write to 15-Courses/<C>/study/guide-<scope>-<date>.md (allowed direct write: new files under study/ only, I1).
- study/practice\_exam.py: generate\_practice\_exam(course, topics, n, difficulty) -> strict JSON schema {questions:[{q, type(mcq|short|problem), options?, answer, explanation, citation}]}; DROP any question whose citation text is not found in the course corpus (I6); render exam.md + key.md; store JSON in db for quiz mode.
- study/grader.py: grade\_attempt(exam\_id, answers) -> score, per-question feedback, weak topics appended to 15-Courses/<C>/study/review-queue.md.
- study/syllabus.py: parse\_syllabus(file) -> extract dated items (exams, assignments, drop deadlines, grading weights) -> suggestion rows (kind=task) with lead-time reminders; NEVER writes tasks directly (I1).
- web/study page: course cards (from 15-Courses/\*/course.md), upload -> materials/ (multipart POST, watcher picks it up), buttons: study guide / practice exam; quiz mode: one question at a time, grade at end, show citations after answering.

EXIT CRITERIA:

pytest -q -> pass incl.: uncited question dropped; syllabus dates -> suggestion rows; grader writes review-queue.md; upload lands in materials/ and is indexed.

Manual: real course PDF deck -> generate 10-question exam -> every question cites a real slide; take quiz in UI -> score + weak topics appear.

## PHASE P2 — Tasks + calendar merge

OBJECTIVE: one merged view of the real day: vault tasks + Google Calendar + Todoist.

BUILD:

- tasks/parser.py: scan vault .md for '- [ ] / - [x]' lines; parse Tasks-plugin emoji markers (due date, scheduled, priority) + #tags + source path/line. Nightly + on-save refresh into db 'tasks' cache table.
- connectors/gcal.py: google-api-python-client + google-auth-oauthlib InstalledAppFlow (Desktop OAuth client; I create credentials.json in Google Cloud Console -- STOP and ask me for it here, this is an expected stop). Token -> keyring. list\_events(range).
- connectors/todoist.py: todoist-api-python, token from keyring; map projects->tags.
- GET /api/agenda?date= -> merged: events + due tasks; GET /api/tasks?bucket=overdue|today|week|someday.
- web/today: timeline (events fixed), top-3 tasks, quick-capture box -> POST appends to 00-Inbox/capture-<date>.md via writer.py. web/tasks: bucket board.

EXIT CRITERIA:

pytest -q -> parser handles due/priority/done/tags fixtures; agenda merges correctly (mocked connectors); quick-capture appends via writer only (grep proof: no other module imports vault write helpers).

Manual: my real calendar + a task in the demo vault appear in one Today view.

PHASE P3 — Planner + suggest-then-approve

```
OBJECTIVE: the agent plans my day; nothing lands without my click.
BUILD:
- agent tools (strict JSON schemas): propose_schedule(blocks[]), propose_task_changes,
  propose_note_edit(path, unified_diff) -> each inserts a suggestion row + rationale.
- agent/prompts/planner.md: inputs = agenda, task buckets, review-queue.md weak topics,
  preferences note 30-Areas/assistant-preferences.md (class schedule, energy pattern,
  focus length). Rules: never move fixed events; study blocks target weak topics near
  exam dates; breaks between blocks.
- writer.py apply(suggestion_id): git commit vault (I2) -> execute: schedule -> gcal
  insert (colorId for FRIDAY blocks); task -> edit task line in source note / Todoist
  API; note -> apply unified diff; append line to 10-Daily/<today>.md '## FRIDAY log';
  mark row applied. dismiss(id, reason) stores reason for planner context.
- web/review: pending cards -- schedule on mini-timeline, note edits as red/green diff;
  Approve/Edit/Dismiss. web/chat: /plan slash-command triggers planner.
EXIT CRITERIA:
pytest -q -> apply(schedule) hits mocked gcal AND vault git log grows by 1 commit;
dismissed reason retrievable; diff apply is idempotent-safe (fails clean
on drifted file).
Manual: '/plan tomorrow' -> blocks on Review page -> approve one -> it appears in
Google Calendar and in the FRIDAY log; git log -1 shows pre-apply commit.
```

PHASE P4 — Briefings + insights + hardening

```
OBJECTIVE: FRIDAY runs itself daily.
BUILD:
- briefing job (APScheduler in backend, 07:00): agenda + due/overdue + yesterday's
  unfinished + exam countdowns + top weak topics -> writes '## Briefing' into today's
  daily note via writer.py + dashboard card. claude-haiku-4-5 for summarization steps,
  opus for the final composition.
- web/insights: task completion trend (14d), overdue heatmap, calendar load vs focus
  time, study streak, practice-exam score trend per course (from db attempts).
- Hardening: request logging of every prompt sent (path list only) viewable at
  /api/audit; global error boundary; `friday doctor` (checks vault, db, chroma, keyring,
  gcal token); README quickstart; Playwright e2e: capture->approve->vault roundtrip.
EXIT CRITERIA:
pytest -q && npx playwright test -> all green.
Trigger briefing manually -> lands in daily note + dashboard.
friday doctor -> all checks OK. /code-review ultra on the full repo; fix CONFIRMED.
```

6. Technology Cheat Sheet

Claude Code should treat these as decisions, not suggestions — they were made once so it never has to re-litigate them mid-build.

| Component        | Choice                    | Implementation notes                                                                                                                                                                                                            |
|------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent runtime    | claude-agent-sdk (Python) | ClaudeSDKClient; model claude-opus-4-8; tools = in-process MCP server (@tool decorator, create_sdk_mcp_server); subscription auth — never set ANTHROPIC_API_KEY (I5); allowed_tools whitelist per entry point (chat vs planner) |
| Embeddings       | BAAI/bge-small-en-v1.5    | sentence-transformers, normalize; ~130MB, CPU fine; first-run model download in `friday init`                                                                                                                                   |
| Vector store     | ChromaDB (persistent)     | PersistentClient('.friday/chroma'); one 'vault' collection; metadata filters course/tags/type; sha1(path+idx) ids                                                                                                               |
| Keyword + fusion | rank_bm25 + RRF           | BM25Okapi over chunk texts (rebuilt on reindex); RRF k=60; then recency decay exp(-age_days/45) on daily/inbox docs                                                                                                             |
| File watching    | watchdog                  | Observer + 2s debounce; ignore .friday/, .git/, .obsidian/, 99-Private/                                                                                                                                                         |

| Component       | Choice                                 | Implementation notes                                                                                                                                     |
|-----------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Doc extraction  | pdfplumber / python-pptx / python-docx | Keep page/slide numbers in chunk meta — citations depend on them (I6)                                                                                    |
| Backend         | FastAPI + uvicorn                      | REST + /ws/chat WebSocket; Pydantic v2 models; CORS localhost:3000 only                                                                                  |
| DB              | SQLite (sqlite3, WAL)                  | Tables: suggestions, tasks_cache, exams, attempts, audit; no ORM needed at this size                                                                     |
| Frontend        | Next.js 14 + TS + Tailwind             | App Router; SWR for data; native WebSocket for chat; recharts for insights; obsidian:// deep links                                                       |
| Google Calendar | google-api-python-client               | InstalledAppFlow desktop OAuth (user supplies credentials.json — expected stop in P2); token in keyring; calendar events insert with a dedicated colorId |
| Todoist         | todoist-api-python                     | Personal API token from keyring; sync read in P2, write via writer.py in P3                                                                              |
| Vault safety    | GitPython                              | Vault is its own git repo (created by friday init); writer.py commits before each apply (I2)                                                             |
| Scheduler       | APScheduler                            | BackgroundScheduler in the FastAPI lifespan; 07:00 briefing; nightly task-cache refresh                                                                  |
| Testing         | pytest + httpx + Playwright            | TestClient for API; tmp vault fixtures; Playwright only for the P4 roundtrip e2e                                                                         |

## 7. Recovery & Maintenance Prompts

### RESUME PROMPT — paste when starting a fresh session

Continue building Project FRIDAY. Read docs/BUILD\_STATE.md for the current position, docs/BUILD\_LOG.md for decisions already made, and the phase prompt in Section 5 of docs/FRIDAY-Autonomous-Build-Playbook.pdf for the active phase. Re-run the test suite first to confirm the baseline is green, then continue from the next unfinished task. All Master Prompt invariants (I1-I6) and the skill workflow still apply.

### STUCK PROMPT — if the same error persists after 2 attempts

Stop patching. Invoke Skill superpowers:systematic-debugging for the current failure. Reproduce it minimally, state your hypothesis before each change, and log root cause + fix in docs/BUILD\_LOG.md. If it's an external blocker (API quota, OAuth, missing credential), record it in BUILD\_STATE.md as BLOCKED with what you need from me, then move to the next independent task instead of waiting.

### PHASE-GATE PROMPT — your reply at each phase end (optional manual gating)

Show me: (1) the exit-criteria output for this phase, (2) git log --oneline for the phase, (3) the /code-review summary and what you fixed, (4) a 5-line demo script I can run myself. Then wait for my 'continue' before starting the next phase.

### Expected human touchpoints

- **P2:** create the Google Cloud OAuth desktop client and drop `credentials.json` where Claude Code asks; approve the browser consent screen once. Provide your Todoist API token (Settings → Integrations).
- **Each phase end (~5 min):** run the demo script, click around, say *continue*.
- **Everything else is Claude Code's job.** If it asks a question the spec or this playbook already answers, reply: *'Decide per the playbook and log it in BUILD\_LOG.md.'*

*This playbook + the v0.2 spec are the complete build input. If the two documents ever disagree, the spec defines WHAT, this playbook defines HOW, and the Master Prompt invariants win over both.*